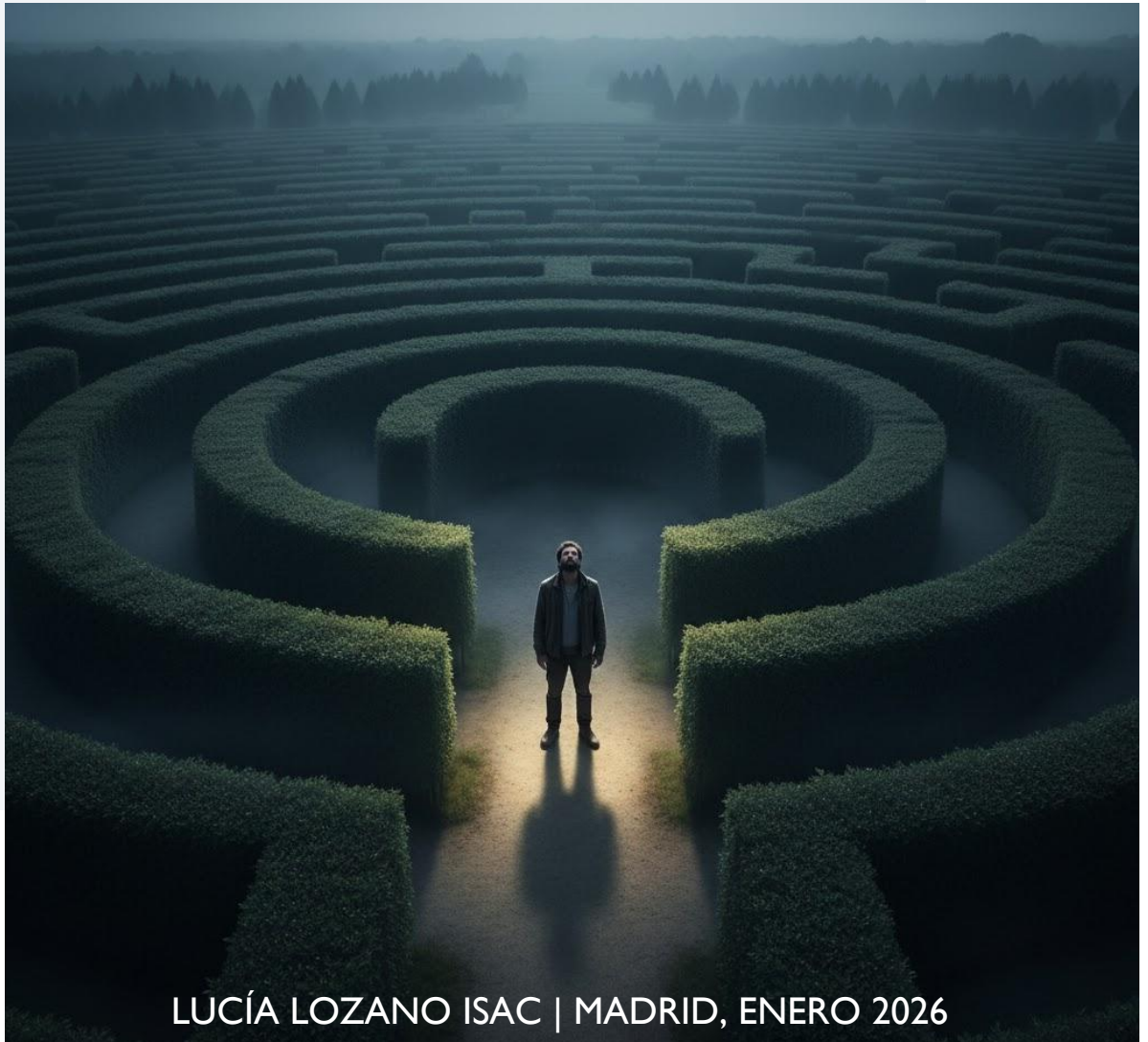


MEMORIA PROYECTO FINAL

FUNDAMENTOS DE LA INTELIGENCIA ARTIFICIAL



LUCÍA LOZANO ISAC | MADRID, ENERO 2026

DIARIO DE MEMORIAS DE VIAJE

Comienzo del viaje.....Página 1

Parte 1 - Recuperación de Kurtz.....Página 5

- PARTE GUIADA
- PARTE CON AGENTES

Parte 2 - Laberinto 2.0.....Página 11

- PARTE GUIADA
- PARTE CON AGENTES

Parte 3 - El final del camino (e inicio del río...).....Página 17

- PROCESO DE DECISIÓN DE MARKOV

Visualizaciones.....Página 21

Final del viaje.....Página 29

Comienzo del viaje

I

Mucho ruido y pocas nueces... ¿o era pocas luces?

Ya habíamos empezado mal el día. La agencia había llamado quejándose de que pasaba mucho tiempo en la selva y poco entre mis queridos coworkers y que debía ir inmediatamente para allá si no quería perder el empleo. ¿Y a mí qué? Si la realidad era que había más animales en mi oficina que en la propia selva. Aunque es verdad, que llevábamos mucho tiempo en “sequía” sin ningún caso y yo ya tenía ganas de acción. Aun así hice lo que hacía siempre con este tipo de llamadas: ignorarlas. Ya se cansarán. Volvieron a llamar y yo a ignorar. Era un bucle sin fin. Tras la tercera llamada, saturado, salgo al porche y me siento en mi hamaca de confianza a observar las olas y a escuchar el silencio. Estaba cerrando los ojos cuando escucho ruido cerca del nogal. Al principio no le di importancia, supuse que sería algún animal. Al cabo de un rato, vuelve a sonar el ruido, pero esta vez presto más atención; es como un serrucho o unos mordisqueos en una rama y fastidiado me levanto a ver que es. En cuanto me acerco, ¡zas! Golpetazo. Miro al suelo para ver qué es y me encuentro a la causante de mi posible y casi segura contusión: una nuez. Sentido tenía al estar debajo de un nogal. Levanto la vista para ver de dónde venía la dichosa nuez, pero solo veo que cae un pedazo de papel y ni rastro de un posible agresor.

*Intrigado, cojo el papel del suelo y leo su contenido: “Mucho ruido y pocas luces, Capitán. Quiere acción, pero ni se levanta del sofá.” Wow, ¿este tipo de qué va? Cuando voy a murmurar una maldición veo que otro papel cae al suelo. De nuevo, levanto la vista, ¿Quién está tirando esto y por qué no veo a nadie? Espero que no sea una broma pesada. Decido darle una oportunidad, la **última**, al que ha interrumpido mi estado de paz. Sin embargo, este papel tiene escrito tres palabras que me dejan pensando: “Coge el teléfono”. Ya no se si esto es real o si alguien se está riendo de mí. Sin embargo, pienso por dentro: “No tengo nada que hacer ni que perder ¿por qué no? Esto es lo más interesante que me ha pasado en los últimos 6 meses.”*

Entro en casa y justo está sonando el teléfono. ¿Casualidad? No lo sé, pero decido cogerlo.

- *¿Estoy hablando con el Capitán Willard?-. dice la voz.*
- *El mismo. ¿Y usted es? -. respondo.*

Ya se lo explicaré más adelante. Me reuniré con usted en el parque del reloj a las seis de la tarde.

Y sin darme tiempo a contestar, la voz colgó. Sorprendido me quedo pensando, ¿qué narices es esto? Pero, sobre todo: ¿quién será el que tiene tanta prisa por reunirse conmigo? ¿Será real?

Solo hay una forma de averiguarlo.

II

El encuentro

18:05 Aquí no veo a nadie. Enfadado y frustrado decido levantarme e irme. Menuda tomadura de pelo. Sin embargo, cuando estoy a punto de recoger mis cosas veo que alguien se acerca.

¿Capitán? -. El hombre alto y moreno me mira serio.

Así es, ¿quién es usted? ¿Es quién me ha llamado esta mañana?

En efecto. Soy Lee Keeley, agente de la CIA y necesitamos que nos ayude a completar una misión. Nos han dicho que es usted el mejor en su trabajo y necesitamos de gente muy preparada. No podemos correr ningún riesgo. Por cierto, siento lo de los mensajes, debíamos llamar de alguna manera su atención.

Está bien que se disculpe porque no ha sido nada agradable recibir esta clase de insultos a primera hora de la mañana. ¿Y cuál es ese trabajo tan peligroso si puede saberse?-. Pregunto incrédulo y cada vez más enfadado. Esta conversación no es lo que me esperaba y la broma ya me está cansando.

Se que no me cree, Capitán, pero tengo pruebas de que lo que le digo es verdad. Si acepta el trabajo tendrá acceso a ellas y descubrirá la veracidad de mis palabras. Hasta entonces, depende de usted dar un salto de fe y confiar, por ahora, en mi palabra.

Bueno, le escucho -. Respondo poco convencido.

No sé si sabrá quién es el Coronel Kurtz. Por si no lo sabe, es un militar de los Estados Unidos, que tras ver los horrores de guerras y homicidios perdió la cordura y se halla en las profundidades de una selva de Camboya, en un templo antiguo. Sin embargo, y, a pesar de su falta de cordura, es pieza clave para ayudarnos en la guerra contra Vietnam, ya que ha dirigido sus ejércitos y sabe todos sus secretos y estrategias militares. Su misión, si decide aceptarla, es encontrar al Coronel y traerlo sano y salvo a la Embajada para que aquí podamos interrogarle y obtener la información que necesitamos. Tiene dos días para darnos una respuesta. En 48 horas le llegará una llamada en la que no escuchará ningún mensaje y su única función es responder sí o no si acepta

o no la misión. Estados Unidos depende de usted -. Y diciendo esto, el agente se marchó.

Sin presión, ¿eh? Pensativo vuelvo a casa. Pero en el fondo tengo clara mi respuesta. Si al final resulta que esto es verdad, puede ser lo que necesitaba: algo de adrenalina y acción. Sin embargo, decido pensarlo bien y consultarlo con la almohada.

III

Empieza la misión

Tal y como dijo Keeley, 48 horas después llega la llamada. Contesto lo que desde un primer momento pensaba contestar: Sí. La llamada se cuelga y al momento suena el timbre. Abro la puerta, pero no veo a nadie, solo un paquete marrón en el felpudo con una nota. Cojo el paquete y leo la nota:

“Hola Capitán

Gracias por aceptar la misión. En el paquete verá que vienen varios libros. Como sabemos que físicamente está bien preparado le enviamos información para que tenga su cerebro preparado también para los desafíos a los que se va a enfrentar. Entre otros libros, le incluimos tres manuales relevantes: Búsqueda lógica Parte 1, El Manual Básico de Inferencia y Luchando contra los MDP. Cabe recalcar que es de vital importancia que se los lea y aprenda. Tiene una semana. Al fondo del paquete encontrará un sobre que contiene billetes de avión hacia Camboya, Si se fija, la ida está programada para dentro de 9 días exactamente, yo me reuniré con usted ahí, y la vuelta para dentro de 39, que si hace los números podemos concluir que tiene como mucho un mes para encontrar al Coronel, ya que a partir de entonces empezará la guerra.

No nos defraude, Capitán. Nuestro futuro depende de usted.

Keeley.”

Trago saliva. “No tengo tiempo que perder” pienso. Así que empiezo a leer los manuales.

IV

Menudo viajecito

Ya estoy en el avión de camino a Camboya. Vuelvo a leer la información que me pasó Keeley sobre el Coronel. Aparentemente y a pesar de su locura, parece un hombre tranquilo. “Eso espero” pienso.

Después de casi 1 día de viaje llegamos a Camboya. Me recibe Keeley y me explica que la idea es hacer noche en un hotel y que a partir de ahí al día siguiente a las 6:00, sale mi tren hacia las afueras de Camboya donde se encuentra el templo. Según el horario llegaré a las 9:00. Keeley me da un walkie para que estemos conectados y me explica que en mi mochila hay un chip rastreador para que pueda saber dónde estoy en cada momento y en caso de necesidad, enviar ayuda .

Necesito descansar, lo sé, va a ser un mes largo, pero no consigo dormirme porque estoy nervioso. No paro de dar vueltas en la cama pensando en si he tomado una buena decisión. En parte estoy contento porque me ha permitido librarme de aquel horrible trabajo de una vez por todas, pero por otra, tengo bastantes dudas acerca de este nuevo reto. Llevo bastante sin ningún caso y no se si he perdido la práctica.

Así, entre vuelta y vuelta llegan las 6:00 y por tanto mi hora de partir.

Con puntualidad británica a las 9:00 entra el tren en la estación, y mientras contemplaba Camboya, lo se: Estoy preparado.

*** A partir aquí empieza un análisis extenso y anecdótico sobre los distintos desafíos a los que me enfrenté en mi misión en Camboya y los peligros y trampas que tuve que superar.*

Parte 1 - Recuperación de Kurtz

En la primera parte, tuve que enfrentarme a un palacio de 36 habitaciones que contenía 3 precipicios, 1 soldado, 1 salida y en una de las habitaciones restantes al Coronel. Yo no tenía información acerca de las posiciones de estos obstáculos ni el Coronel, pero sí tenía pistas, ya que, si estaba en habitaciones contiguas, sentía una brisa, escuchaba un ronquido, sentía el tacto de las paredes o veía un resplandor cuando en una habitación adyacente a la que estaba, había un precipicio, un soldado o la salida respectivamente. Además, contaba con una granada que podía tirar cuando pensara que el soldado estaba en una habitación. Si acertaba, el soldado moría, escuchando un grito y podía pasar a esa celda, si fallaba, el soldado seguía vivo y había desperdiciado mi granada. Si entraba en una sala con precipicio o soldado (cuando el soldado estaba vivo) moría al instante y significaba el fracaso de mi misión. Además, cada obstáculo y elemento se colocaba de manera aleatoria en el palacio, por lo que cada vez que entraba debía volver a elaborar de nuevo mis mapas. Suerte que en el manual venía información sobre búsqueda lógica porque me permitió realizar la búsqueda de dos maneras: guiándome por mis instintos (el usuario guía) o basándome en un agente de búsqueda (BFS). Lo que sí simplificó esta parte de la búsqueda fue que solo podía haber un elemento por habitación. Uf, menos mal. Aunque no sabía lo que se me venía encima...

Podía tomar tres tipos de acciones en función de la situación. Moverme arriba, abajo, derecha, izquierda. Tirar la granada si el soldado estaba en una habitación contigua o sabía con certeza donde se encontraba. Salir, una vez encontrados el Coronel y la salida, ya podía salir del palacio. Si escogía moverme en cada movimiento solo podía pasar a una de las habitaciones contiguas (distancia Manhattan = 1). En cada habitación deducía información sobre las habitaciones seguras, posibles ubicaciones de precipicio y salida, basándome en los preceptos que recibía en esa habitación.

Por supuesto, cuando encontré a Kurtz, no tuve ningún problema para convencerle de que se viniera conmigo.

Keeley y yo nos dimos cuenta de que el palacio estaba programado con un gran código de Python, así que como él es el experto, os dejo con él para que os haga un análisis sobre el funcionamiento interno del palacio

¡Hasta la próxima!

PARTE GUIADA

El código presenta una parte guiada en la que el usuario dirige al Capitán basándose en sus deducciones lógicas. Empezamos imprimiendo un menú explicativo y preguntando con un input que opción quiere el usuario que se lleve a cabo. Si el usuario pulsa la G/g empieza la parte guiada.

La parte guiada consta de 7 funciones auxiliares, de las cuales algunas se usarán también en la parte con agente. Empezamos con la función `creacion_tablero` que recibe como input la dimensión del tablero (6) e inicializa un tablero lleno de *. Para colocar los elementos, se ha creado una función `añadir_elementos`, que toma como input la dimensión, el tipo de elemento, cuántos elementos hay de ese tipo y el tablero del que parte. Para que las posiciones no se colapsen y guarde el progreso anterior. La función `añadir_elementos` devuelve un tablero con el elemento correspondiente añadido, que estará en una posición distinta a la (0,0) y siempre en otra en la que no haya otro elemento colocado previamente. La función `creacion_tablero` va almacenando los cambios y cuando ya se han añadido todos los elementos devuelve un tablero final, que será el que define el palacio y el que usaremos en la partida.

La función `vecinos` recibe como input la fila, columna y dimensión, de la habitación y devolverá en una lista las habitaciones adyacentes a dicha habitación, teniendo en cuenta los casos en los que esté en la primera fila, la última, primera columna y última columna.

`obtener_perceptos` recibe como input el tablero, la fila, la columna, la variable grito y la dimensión y se encarga de devolver una lista con los perceptos correspondientes a la habitación que estamos analizando. Empezaremos obteniendo las habitaciones contiguas con la función `vecinos` y analizaremos si en alguno de los vecinos hay algún precipicio, en cuyo caso, `brisa = True`, si no ha habido grito y si hay algún soldado en los vecinos, en cuyo caso `ronquido = True` y si la celda en la que estamos es la salida o algún vecino es, en cuyo caso `resplandor = True`. También analizamos si está en alguna pared, viendo si estamos en la primera o última fila o si estamos en la primera columna o última, en cuyo caso las paredes correspondientes serían `True`. Finalmente devolvemos la lista con los perceptos.

La función `validación_celda` recibe como input la fila, la columna, la dimensión, el tablero, la lista de seguras, las listas de posibles soldados, precipicios y salidas, los movimientos posibles, la variable grito, la variable granada y la lista de memoria y se encarga de analizar la habitación en base a los perceptos que recibimos en esa habitación, imprimiendo esa información por pantalla (deducciones del Capitán). Vemos también si no hemos visitado esa celda, si no es así, la añadimos a la lista de memoria y si no estaba en la lista de celdas seguras, la añadimos también a esa lista. A partir de aquí veremos si hay algún elemento cerca de la habitación. Por ello anteriormente obtuvimos también los vecinos. Si no hay brisa, entonces si esa celda estaba en la lista de posibles precipicios, será eliminada al no considerarse ya celda peligrosa. Si recibimos brisa, indicamos en un mensaje que hay un precipicio cerca y si los vecinos no están en celdas seguras (porque ya los hayamos inspeccionado) ni en la lista de posibles precipicios, los añadimos a dicha lista (`posible_precipicio`). Lo mismo haremos para el soldado. Si escuchamos un grito, el soldado ya ha caído y ya no nos hará falta la lista de posible_soldado.

Si no lo escuchamos, pero escuchamos un ronquido, indicamos en un mensaje que hay un soldado cerca y si los vecinos no están en la lista de habitaciones seguras ni en la de posible_soldado, la añadimos. Si no recibimos el estímulo y la celda estaba en posible_soldado la eliminamos de ahí, ya que ya no es peligrosa. La misma lógica se da para la salida. Esta forma de haberlo programado tiene también en cuenta el caso en el que el soldado y algún precipicio estén en habitaciones contiguas a la celda que estamos analizando, añadiendo los vecinos en ambas listas (no elif sino if). Si no recibimos ningún estímulo lanzamos un mensaje por pantalla indicando que estamos en una celda segura. Pasaremos a analizar si hay paredes en la posición en la que estamos obteniendo así los movimientos reales en ese movimiento. Finalmente devolvemos una lista con los movimientos posibles, la lista de seguras y de los posibles. Como observación los mensajes que se imprimen por pantalla acerca de la cercanía de algún peligro son de utilidad para el usuario, ya que al no tener conocimiento de lo que se halla debajo de los * le permite hacerse una idea de lo que se esconde y de donde están ubicados los peligros, o si no se sabe con certeza la habitación porque aún no se tiene toda la información necesaria, por lo menos la zona.

La función mostrar_estado_juego recibe como inputs la lista de celdas seguras, las de los posibles, el parámetro salida_encontrada, el parámetro grito, la lista de habitaciones visitadas y el tablero y muestra las celdas visitadas y seguras hasta el momento. Aporta información acerca de posibles ubicaciones de la salida si aún no se ha encontrado (parámetro salida_encontrada). Sobre las posiciones del soldado, si no se ha escuchado un grito (parámetro grito) ya que, si no, el soldado ya ha sido eliminado. Si la longitud de la lista de posible_soldado es 1 quiere decir que el soldado está en esa posición, lanzamos un mensaje indicando que ya sabemos dónde está el soldado y su posición. El mismo proceso se sigue para los precipicios. Imprimir esta información (conclusiones del Capitán) facilita el uso al usuario, para no trabajar sobre abstracto y poder ubicar toda la información que tiene sin perderse.

La función movimiento recibe como inputs, la fila actual, la columna actual, el tablero, la dimensión del palacio, la lista de seguras, los posibles, los parámetros granada, grito, salida_encontrada, kurtz_encontrado, y un tablero visible que en la siguiente función se explicará su uso y se encarga de llevar a cabo todo lo relacionado con el movimiento del Capitán. Definimos los movimientos posibles y llamamos a la función validación_celda, de la que obtendremos, los movimientos reales, y la lista de seguras y posibles actualizadas. Mostraremos el estado actual del juego con la función mostrar_estado_juego y pediremos por pantalla un movimiento al usuario indicando cuáles son posibles en esa habitación, habiendo verificando antes si podemos añadir la acción de la granada a los movimientos, viendo si hay granadas, si el soldado no ha caído y si no está ya añadido a la lista de movimientos posibles, para ello primero verificamos si recibimos el estímulo de ronquido, en cuyo caso podremos añadir la acción de tirar granada a los movimientos posibles. Imprimimos cuáles son los movimientos posibles en esa habitación y pedimos un movimiento por pantalla. Si ese movimiento es válido y se encuentra entre los posibles, ejecutamos la acción, si no, volvemos a pedir un movimiento hasta que este sea válido. Almacenamos para su posterior uso la posición actual. Pasamos a procesar el movimiento. En función del tipo de movimiento

actualizaremos la fila o la columna (W, A, S, D → arriba, izquierda, abajo, derecha respectivamente). Para el movimiento de la granada hemos implementado una doble comprobación de validez verificando si no hay granadas, en cuyo caso lanzamos un mensaje de invalidez y volvemos a pedir otro movimiento. En caso de que la granada esté disponible, se muestran las celdas disponibles para tirar la granada (adyacentes a la actual y con un input se pide al usuario que seleccione una, teniendo en cuenta la información que tenemos acerca del soldado y el usuario meterá una celda con formato fila, columna. Si la celda no es válida por no ser adyacente o hallarse fuera de los límites del tablero, se pide otra posición al usuario. Cuando la posición es válida, verificamos si esa celda correspondía con el soldado, en cuyo caso, lanzamos un mensaje positivo, y si no una indicación de falta de granadas y la no presencia del soldado en esa celda. Después de los movimientos/acciones actualizamos en ambos tableros la antigua posición (de ahí que antes la almacenáramos) a * y verificamos la nueva posición. Si coincide con un obstáculo letal se acabó el juego, lanzamos un mensaje de GAME OVER. Si no, devolvemos la fila actual, la columna actual, el estado de la granada, del grito, obstáculo_letal (indicador de la continuación de la partida) y de los posibles. Con esto terminamos las funciones auxiliares de la parte de juego guiado.

La función principal de esta parte recibe como nombre juego_guiado y como argumentos la dimensión del palacio. Se encarga de juntar todas las piezas del puzzle. Comenzamos creando dos tableros, el que representará el entorno del palacio (tablero_final) y el que será visible al usuario para ubicar su posición en el tablero. Este tablero visible solo tendrá * y la posición del Capitán Willard (CW) y a medida que se vayan recorriendo las celdas se actualizará con X las celdas visitadas. Esto facilita que el usuario no se tenga que imaginar donde se sitúa en el tablero, ya que, aunque se den las coordenadas actuales es más fácil verlo visualmente. En esta función, además, se inicializan todas las listas y variables de las que se hará uso en la partida. Situaremos a Willard en la posición inicial (0,0). Como relevante cabe resaltar salida_pos que almacenará la posición de la salida si la encontramos durante el trayecto y obstáculo_letal que nos indicará si la partida termina de manera abrupta (soldado o precipicio) o si podemos seguir jugando. Nuestro primer while (FASE 1) tiene como condiciones que no se haya encontrado ni un obstáculo letal ni a Kurtz, ya que se encargará de ir recorriendo el tablero en busca del Coronel. Ejecutaremos la función movimiento de la que como se explicó recibimos como output la fila actual, la columna, granada, grito, obstáculo_letal y los posibles. Si obstáculo_letal = True, se acabó la partida, si no, seguimos. Si la nueva celda es la salida y todavía no la habíamos encontrado (es decir, no volvemos a pasar por la misma casilla, es la primera vez que caemos en ella), lanzamos un mensaje de indicación y en salida_pos guardamos su posición. En el tablero visible, ponemos esa posición con una F (final). Si encontramos a Kurtz, lanzamos un mensaje, Willard y Kurtz pasan a viajar juntos y salimos del bucle de la FASE 1.

Si hemos encontrado a Kurtz y no nos hemos topado con un obstáculo peligroso pasamos a la FASE 2: la búsqueda de la salida. Si ya la habíamos encontrado, se acabó el juego, ¡hemos ganado! Si no, debemos buscarla. Empieza el segundo while. El proceso es el mismo que para

el primer while, pero ahora solo nos centraremos en no encontrarnos ningún obstáculo y encontrar la salida. Si encontramos la salida y no un obstáculo letal, empieza la FASE 3: ir a la salida. De nuevo nuestra preocupación es no encontrar ningún obstáculo letal. El proceso lógico de esta última fase es ir moviéndonos por el tablero evitando los obstáculos e intentando llegar a la salida (que ya sabemos su posición), o sea el desafío aquí es no cruzarse ningún obstáculo en nuestra vuelta a la salida. Si llegamos, ejecutamos la opción salir y salimos del palacio. ¡Ojo! Esto solo puede suceder si ya hemos encontrado al Coronel.

PARTE CON AGENTES

En esta opción el Capitán sigue una búsqueda siguiendo el agente BFS con estructura de cola (FIFO → First In First Out). Esta opción se da porque el usuario pulsa A/a en el teclado.

La parte con agentes se compone de 5 funciones auxiliares, pero hará uso también de las funciones creación_tablero y por tanto añadir_elementos, explicadas anteriormente.

La primera función auxiliar de esta sección es bfs_camino que recibe como input la fila, columna, la lista de habitaciones visitadas, la lista de seguras, la dimensión del tablero, el tablero y el parámetro grito y se encarga de buscar un camino y analizar las celdas seguras. Empezaremos creando una cola (si no devolvemos None) a la que añadiremos la celda y una lista vacía que será el camino y un conjunto de estados explorados (set). Mientras sigamos teniendo elementos en la cola, vamos a hacer popleft para obtener una celda y el camino correspondiente a dicha celda. Si ya hemos visitado la celda y sabemos que es segura, no nos interesa analizarla otra vez así que directamente devolvemos el camino. Si no, extraeremos sus vecinos y veremos si son seguros o no. Si lo son, veremos, a modo de comprobación que esa celda no contenga ningún obstáculo letal. Si todo está bien y el vecino que estamos analizando no está en los estados explorados, lo añadimos y a la cola añadimos el vecino y al camino le concatenamos el vecino (mejor que hacer append para evitar modificaciones del camino).

La siguiente función para analizar es bfs_salida que recibe como argumentos la fila, columna, salida_pos (posición de la salida), la lista de seguras, la dimensión del tablero, el tablero y el parámetro grito y se encarga de recorrer el tablero para llegar la salida. La usaremos una vez se haya encontrado a Kurtz. Esta función se ha implementado igual que la función bfs_camino, pero ahora su objetivo es ver si la celda que extrae de la cola coincide con la posición de la salida, para así devolver el camino y terminar el juego.

La función razonamiento_agente lleva a cabo todo lo relacionado con el proceso lógico, representa lo que en parte guiada era validación_celda, pero ahora no podemos aprovechar esa función porque el agente solo necesita cierta información ni le hacen falta los mensajes de alerta que enviábamos antes al usuario. Por limpieza y organización y evitar posibles errores, he diseñado una función que, aunque en bastantes cosas se parece a validación_celda en otras no y son esas cosas la clave para el buen funcionamiento de esta parte del proyecto.

Como he dicho, es muy similar a validación_celda así que me centraré en explicar las diferencias. En esta función no es necesario analizar los movimientos posibles ni centrarnos

en si nos encontramos con alguna pared. La lógica para actualizar las celdas seguras es mucho más simple que en `validación_celda` y esta función devuelve la lista de seguras, los posibles, y una lista con los estímulos brisa, ronquido, resplandor.

La función `movimiento_agente` es también muy similar a la función `movimiento` de la parte guiada solo que suprimiendo las entradas por teclado y las decisiones del usuario. Recibe como inputs la fila actual, columna actual, tablero, la dimensión, la lista de seguras, los posibles, los parámetros granada y grito, la lista de celdas visitadas y el tablero visible. Esta función se encarga de realizar los movimientos del agente. Comenzaremos viendo si ya hemos visitado o no esa habitación añadiéndola en caso negativo a la lista de visitadas. Obtenemos la actualización de la lista de seguras y los posibles con la función `razonamiento_agente` y comprobamos si la posición actual coincide con la salida o con el Coronel, en cuyo caso lanzamos un mensaje de indicación por pantalla. Si encontramos al coronel directamente devolvemos la fila actual, la columna, el estado de la granada y de grito, `obstáculo_letal = False`, `objetivo_alcanzado = True` y la lista de seguras y posibles actualizada. Si no encontramos ni la salida ni al coronel empieza la búsqueda.

Creamos una lista auxiliar de celdas seguras y obtenemos el camino a seguir llamando a la función `bfs_camino`. Si nos devuelve un camino, avanzamos una posición a la siguiente celda (`camino[0]`), actualizamos el tablero visible, con una X y la fila y columna a la fila y columna de la celda siguiente. Verificamos si la celda siguiente equivale a algún soldado vivo o precipicio devolviendo en cuyo caso la fila actual, la columna, el estado de la granada y de grito, `obstáculo_letal = True`, `objetivo_alcanzado = False` y la lista de seguras y posibles actualizada. Si `camino = None`, para evitar que el agente se quedara bloqueado y no pudiera avanzar más ante la inestabilidad de celdas inseguras, he implementado una continuación donde Willard se atreve a avanzar a una celda no segura (lógicamente, hay veces que el agente se encuentra con un obstáculo, pero otras no, por eso pensé que podría ser interesante tomar ese riesgo, porque hay posibilidades de encontrar al Coronel y la salida). Sin embargo, aún así, hay veces que se queda completamente atascado, lo cual indicamos con un mensaje, tomándolo como si se hubiera encontrado con un obstáculo para que no se produzca un bucle infinito y no esté dando vueltas sin control.

Finalmente, como última función auxiliar tenemos `obtener_ruta_optima_kurtz` que se encarga de encontrar el camino óptimo desde el origen hasta Kurtz. Esta función se ejecuta una vez hemos encontrado a Kurtz y tenemos su posición. Recibe como argumentos la posición de inicio, la de Kurtz, la lista de seguras, la dimensión del tablero, el tablero, y el parámetro grito. De manera similar a como hacíamos `bfs_camino` o `bfs_salida` buscamos el camino a Kurtz (fin) y si lo hay lo retornamos.

La función principal recibe el nombre `juego_con_agente_bfs` y como argumento la dimensión del palacio. Tal y como hacíamos en `juego_guiado`, inicializamos, tablero, variables y listas y definimos 3 fases; encontrar a Kurtz, encontrar la salida, si no la habíamos encontrado e ir a la salida cuando la conozcamos. La única diferencia es que empleamos las funciones del agente en vez de las de usuario.

Parte 2 - Laberinto 2.0

Aliviado por fin, suspiro, hemos conseguido salir del palacio. Le dije al Coronel que me sentaría un momento si no le importaba a descansar, mis neuronas y mi cuerpo estaban exhaustos. El Coronel dijo que él daría una vuelta por los alrededores y para evitar que se perdiera le di un walkie para que se pudiera poner en contacto conmigo en caso de que le pasara algo. El Coronel me dijo que no me preocupara y que en media hora volvería conmigo y seguiríamos con la misión. Intenté no dormirme, pero al final, al final... El sueño me pudo y cuando desperté el Coronel no estaba y ya era de noche, a veces recorrí los alrededores llamándole para ver dónde estaba, pero nada, silencio absoluto, hasta que de repente sonó el walkie: "Capitán... perdido ... templo". Fueron las únicas palabras que pude captar. No me fastidies. Se ha perdido en el templo. Así pues, sin más opción me dirijo al templo de Teká en busca (de nuevo) del Coronel.

Cuando llegué al templo, para mi terror, era un laberinto 2.0 del palacio anterior, solo que con otras trampas. En concreto había una trampa de fuego, otra de pinchos y otra con dardos venenosos, además de otro soldado roncando. Además, para mayor sorpresa, estas trampas podían estar en la misma habitación y el soldado podía estar en la salida o incluso con el Coronel. Lo bueno era, que, como antes tenía alguna pista sobre la ubicación de estas trampas. Si me llegaba un olor a queroseno, cerca estaba la trampa de fuego, si el suelo crujía, la trampa de pinchos estaba por ahí, si veía unos cables, no cabía duda de que la trampa de dardos estaba al lado. Por otra parte también, como antes podía escuchar el ronquido del soldado si estaba en una de las habitaciones contiguas y el destello de la salida si esta estaba cerca. Sin embargo, esto no era como el palacio anterior, aquí tenía que poner en práctica lo que había aprendido de El Manual Básico de Inferencia, ya que con las probabilidades a priori de que en cada celda hubiera uno de los elementos, pude modelar un modelo de verosimilitud que valía 1 si la celda adyacente contenía un estímulo y 0 si no (la propia celda también estaba incluida en el modelo). Fui recorriendo el templo usando la regla de Bayes y conseguí el posterior normalizando la multiplicación de prior x verosimilitud.

Menos mal que tenía mis mapas para poder orientarme...

De nuevo, te dejo con Keeley, que te explicará con mayor profundidad lo que viví en aquel tenebroso templo...

¡Hasta la próxima!

PARTE GUIADA

De nuevo se presenta al usuario la opción de escoger entre una opción guiada y otra con agentes, empleando esta vez el agente A*.

La parte guiada se compone de 13 funciones auxiliares, de las cuales usaremos varias para la parte con agentes. Las primeras funciones, `creación_tablero`, `añadir_trampas`, `añadir_resto_elementos`, siguen la misma lógica que en la parte 1 pero esta vez adaptado a lo que tenemos. Definiremos el tablero con listas vacías a las que iremos añadiendo los elementos. Primero añadiremos las trampas, que menos en la posición inicial pueden estar en cualquier otra posición. Los otros elementos se añaden después con la condición de que no estén en la posición o en una posición donde haya una trampa.

De la parte 1 rescatamos la función de vecinos. La función `obtener_perceptos` la hemos implementado de manera muy similar a la parte 1. Solo que esta vez, como dijo el Capitán, el estímulo también puede estar en la propia celda así que veremos si hay estímulos en los vecinos y en la propia celda de cada una de las trampas. Para el soldado comprobamos también su estado de vida (grito) y obtenemos también las paredes devolviendo una lista de perceptos. A partir de aquí empiezan las funciones propias de esta parte

La función `probabilidades_priori` recibe como argumento la dimensión del templo. Tal y como especificaba el enunciado, definimos el prior y partimos pensando que todas las celdas tienen la misma probabilidad de tener una trampa, salida, soldado o Coronel Kurtz. Inicializamos un tablero con todas las posiciones con probabilidad prior excepto la posición origen (0,0) en la que está el Capitán, que ya sabemos que es segura. Creamos un tablero para cada posible estímulo; trampa de fuego, pinchos, dardos, soldado, salida y Coronel y devolvemos el diccionario. Devolver un diccionario con una matriz para cada estímulo va a ser útil a la hora de mostrar los mapas y actualizar las probabilidades de cada uno, así no se mezclan los datos.

La función `probabilidades_likelihoood` recibe como inputs la dimensión, la fila, la columna y el parámetro percibido y se encarga de definir el modelo de verosimilitud. Crea una matriz inicializada a 0.0 y selecciona los vecinos y la propia celda de la celda que se está evaluando. Si `percibido = True`, entonces pone en la posición de las celdas relevantes el valor de 1.0, si no recibimos el estímulo, las celdas relevantes tienen su valor a 0.0 y son las demás las que tienen su valor a 1.0, tal y como se define la verosimilitud en el enunciado. El paso de poner los valores a 1.0 de las celdas no relevantes si no recibimos el estímulo es fundamental para el posterior desarrollo del proyecto.

La función `probabilidades_posterior` recibe como inputs la matriz del prior, la fila, la columna, la dimensión y el parámetro percibido. Esta función se encarga de crear las matrices posterior con los datos recopilados, por ello obtiene la matriz de verosimilitud de la función `probabilidades_likelihoood` e inicializamos lo que será la matriz del posterior a 0.0. Aplicamos la regla del posterior: $\text{prior} * \text{likelihood}$ y multiplicamos las matrices calculando la norma que será la suma de los elementos de la nueva matriz. Para que esté normalizada la norma tiene

que ser mayor que 0, por lo que, si no lo está, la normalizamos dividiendo a cada elemento entre la norma y tras esto, devolvemos la matriz posterior normalizada.

La función `mapas_de_calor_probabilidades` recibe como argumentos el diccionario de estímulos con valor las matrices del prior, la dimensión del templo, la fila y la columna y se encarga de mostrar los mapas de calor (heatmaps) de las probabilidades de encontrar alguna trampa (F, P, D), soldado (M) o salida (S). Para el mapa de calor de las trampas debemos sumar las probabilidades en cada posición de cada trampa, excepto de la celda actual, que sabemos que es 0.0. Con esto calculado, representamos usando matplotlib y seaborn los heatmaps y a cada uno le asignamos un color (fórmula obtenida de la Chuleta de Probabilidad y Estadística (1º iMat)).

La función `movimientos_seguros` recibe como argumentos el diccionario de estímulos con valor las matrices del prior, la fila, la columna, la probabilidad umbral y la dimensión y se encarga de, basándonos en las probabilidades y los estímulos de designar si una celda es segura usando la probabilidad umbral. Se considera que una celda es segura si la suma de todos los riesgos en esa celda es menor que la probabilidad umbral. Por otra parte, si hay soldado, buscará de los vecinos cuál es la celda que tiene mayor probabilidad y la añadirá a `obj_granada`, actualizando la probabilidad máxima, pudiendo haber varios elementos en dicha lista.

La función `realidad` se encarga de designar el estado del juego basándose en la celda en la que se encuentra el Capitán en ese momento, recibe como argumentos la fila, la columna, el tablero, los parámetros kurtz que indica si Kurtz se encuentra o no en la celda y grito y las tuplas salida que representa, si las hay, las coordenadas de la salida y la tupla `pos_kurtz` que representa, si las hay, las coordenadas donde se encuentra el Coronel. Verificamos si la celda contiene alguna trampa o algún soldado vivo, en cuyo caso devolvemos “MUERTE” y las posiciones de la salida y kurtz a None. Si encontramos a Kurtz almacenamos su posición y `kurtz = True`. Si además encontramos la salida en esa posición, lo tenemos, devolvemos “VICTORIA”, `kurtz = True` y las posiciones de la salida y kurtz. Si no encontramos la salida o la encontramos y no está kurtz devolvemos “VIVO”, `kurtz = False`, la posición de la salida (si la encontramos) y la de kurtz a None.

La función `movimientos_posibles` recibe como argumentos la lista de perceptos y el parámetro granada y en función de los perceptos que reciba designa si un movimiento es posible o no, teniendo en cuenta paredes y soldado, devolviendo la lista de movimientos posibles en dicha celda.

La función `validar_ejecutar_movimiento` sería muy similar a la función `movimiento` que teníamos en la parte 1, ya que recibiendo como inputs la fila, la columna, los parámetros granada, grito, estímulo de soldado y las listas perceptos, `blanco_granada` y tablero, se encarga de pedir por pantalla un movimiento y validarlo. La diferencia radica en la acción de tirar la granada, donde si no recibe estímulo de ronquido, indica que no hay estímulo y que por tanto no se tira la granada, si hay estímulo y posiciones para tirar la granada, si la posición es única, directamente tira la granada a la celda que se encuentra en `blanco_granada`. Si `blanco_granada` tiene más de una posición porque varias celdas tienen la misma probabilidad, tal y como

hacíamos en la parte 1, ofrecemos al usuario opciones para tirar la granada y una vez se haya introducido la celda correctamente se verifica si la granada ha acertado (`grito = True`) o si, por el contrario, no había soldado en dicha celda. Finalmente, esta función devuelve la fila, columna (modificadas por cualquier otro movimiento que no sea G), el estado de la granada y grito.

La función principal, como en la primera parte, recibe el nombre de `juego_guiado` y recibe como argumento la dimensión del templo. De nuevo, siguiendo el proceso que seguía `juego_guiado` en la parte 1, inicializamos las variables incluyendo una probabilidad umbral que establecimos a 0.25. En la primera fase, imprimimos el tablero visible y obtenemos los perceptos, centrándonos en el del soldado, así como las matrices del prior por separado. Buscaremos las celdas seguras y las posibles celdas para tirar la granada con la función `movimientos_seguros`. Mostramos los mapas de calor y actualizamos el tablero visible mostrando una recomendación de celdas a seleccionar (celdas seguras). Si recibimos el estímulo de soldado indicamos las celdas en las que podría estar y ejecutamos la función `movimiento` usando la función `realidad` para validar el resultado de ese movimiento. Si el resultado es MUERTE, hemos topado con un obstáculo letal y hemos muerto, si es “VICTORIA” hemos completado con éxito la misión. Las otras dos posibilidades es que sea una celda que no contenga ningún elemento, por lo que seguimos el bucle de manera normal, y, que encontremos a Kurtz, en cuyo caso pasamos a la fase 2. La fase 2 se ejecutará igual que la fase 1 pero ahora nos centraremos en el percepto de salida y será lo que busquemos, concluyendo únicamente en dos estados: MUERTE, si topamos con un obstáculo o VICTORIA si encontramos la salida (ya habíamos encontrado a Kurtz).

PARTE CON AGENTES

La parte con agente se ejecuta cuando el usuario pulsa las teclas A/a. Su estilo es similar a la parte 1, pero esta vez hemos empleado el agente A*. Agente de búsqueda que emplea una función de evaluación basada en una heurística y un coste, para decidir sus movimientos.

Se compone de 6 funciones auxiliares propias, aunque también usa funciones definidas en la parte guiada.

La función heurística obtiene la distancia Manhattan entre dos celdas, que usaremos como heurística en la función de evaluación. La función `coste_acumulado` se encarga de calcular el coste acumulado de la celda como suma de riesgo de todas las trampas y el soldado y evaluando. Si este riesgo es mayor o igual a 0.95 (decidido para la decisión de los movimientos y facilidad para el agente A*) entonces devolvemos infinito, como indicador de que en esa celda casi seguro hay una trampa y no debemos entrar en ella. En caso contrario sumamos 1 (coste de un paso) al riesgo de encontrarnos alguna trampa o soldado multiplicado por 5 (penalización suave para que no se quede dando vueltas entre las dos mismas celdas todo el rato) y devolvemos ese valor.

La función `a_estrella_algoritmo` se encarga de llevar a cabo el algoritmo A*. Recibe como argumentos el origen, el objetivo, el diccionario de estímulos con valor las matrices y la dimensión del templo. Crearemos una lista que será la frontera, y una cola de prioridad (heapq) que guardará los nodos a explorar y explorados. Los diccionarios `origen` y `g_real` contienen el padre del nodo a evaluar y los costes de los nodos inicializados a infinito respectivamente. Para evitar bucles infinitos estableceremos un número máximo de iteraciones que será $\text{dim}^2 * 10$ que sería como pasar por cada celda 10 veces. Siempre que la frontera no esté vacía y que las iteraciones sean menores que el número máximo de iteraciones extraeremos el nodo actual y evaluaremos si ese nodo ya lo hemos explorado o no añadiéndolo al set cerrados en caso de que no. Si el nodo actual es el nodo objetivo, llamamos a la función `camino` (que ahora definiré) retornando así el camino del origen al destino. Si no se da este caso, calculamos los vecinos del nodo actual y veremos si los hemos evaluado o si no. Calcularemos el coste actual del vecino que estamos evaluando y si su coste es infinito, porque sea trampa o no sea celda segura, no nos vale, en caso de que esta situación no se dé, el coste acumulado del vecino es sumar el coste acumulado de la posición de `nodo_actual` y el coste del vecino. Si el coste acumulado es menor que el que el vecino tenía en el diccionario de costes, actualizamos coste y ponemos al vecino como padre del nodo actual. Finalmente haremos uso de la heurística y calcularemos el valor total de la función evaluación sumando el valor de la heurística en la posición del vecino y el coste acumulado del vecino añadiendo a la cola de prioridad la frontera y el vecino con su función de evaluación. Si no hubiera frontera devolvemos `None`.

La función `camino` calcula el camino desde el inicio al objetivo haciendo uso de los inputs `origen` (diccionario de padres e hijos) y el nodo actual, que será el objetivo y devuelve ese camino invertido, para que vaya de inicio a fin, ya que el camino lo recorremos al revés (desde el objetivo al inicio).

La función `obtener_coronel_aprox` recibe el diccionario de priors, de celdas visitadas y la dimensión del templo como parámetros y devuelve una tupla de lo que se cree que es la posición de Kurtz. Evaluaremos cada celda del tablero viendo primero si ya la hemos visitado, en cuyo caso añadimos una penalización, si no, la penalización es 0. Definimos el objetivo a `None` y el mejor valor como infinito. Calcularemos el riesgo total de las trampas y el soldado y la probabilidad de encontrar a Kurtz en dicha celda. Si esta es mayor que 0.6 (estipulado para facilitar la búsqueda del objetivo), devolvemos las coordenadas, si no, calculamos la diferencia del riesgo frente a la probabilidad de encontrar a Kurtz multiplicada por 3 (para reducir el valor y por tanto posibilidad de ir hacia esa celda) sumándole la penalización. Si este valor es menor que `mejor_valor`, actualizamos `obj` y `mejor_valor` hasta que devolvemos finalmente la tupla de coordenadas de las que se cree que se encuentra Kurtz.

La función `obtener_salida_aprox` se encarga de hacer exactamente lo mismo que la función anterior, pero en vez de buscar la posición de Kurtz, busca la salida. Esta implementación reduce ir a celdas poco probables y por tanto hace que sea más eficiente y rápido.

La función principal recibe como nombre `juego_con_agente_A_estrella` y como argumento la dimensión del templo. Esta función sigue la misma lógica que la función `juego_guiado` de la parte 2. Inicializa las variables, incluido el diccionario vacío `visitadas`, y obtiene los priors, manteniendo como referencia la probabilidad umbral a 0.25. En cuanto a diferencias respecto a la función `juego_guiado` están que en este caso obtenemos la posición aproximada (si la hay) del coronel y empleamos el algoritmo `a_estrella_algoritmo` para obtener una ruta. Si existe ruta, actualizamos valores y nos movemos a la siguiente celda (avanzamos un paso). Si no obtuviéramos una ruta, volvemos a intentar calcular de nuevo una ruta, pero aumentando 0.25 la probabilidad umbral, para ver si con eso conseguimos una ruta. En caso de que no, se acaba el juego, el Capitán está atascado definitivamente. En caso de que sí, actualizamos posiciones y valores y proseguimos a las conclusiones. Empleamos la función de realidad explicada en la parte guiada para concluir si el Capitán se ha topado con un obstáculo letal o si sigue vivo y puede continuar su aventura. Si encontramos al Coronel, es momento de pasar a la fase 2: la búsqueda de la salida. Si la salida ya la habíamos encontrado por el camino se acaba el juego, lo hemos conseguido. Si no hemos encontrado la salida, volvemos a ejecutar lo mismo que hicimos en `juego_guiado` pero buscando la salida y concluimos en MUERTE si nos topamos con un obstáculo o VICTORIA si encontramos la salida.

Mientras que BFS busca el camino más corto en pasos, A busca el camino más seguro minimizando el riesgo acumulado.

Parte 3 - El final del camino (e inicio del río...)

Finalmente, después de 10 arduos días, encontré al Coronel, se había quedado encerrado en una habitación y no podía salir... Desde ese momento decidí que no le perdería de vista ni un solo momento. Qué bien me vinieron los manuales que me pasó Keeley, menos mal que me acordaba de todo... Nos enfrentábamos ahora al eslabón final de nuestra misión. Atravesar el río Nung, que por sí no lo sabíais está repleto de peligros. Está plagado de corrientes ocultas que te pueden llevar de una punta a otra del río sin que te des cuenta, son muy **aleatorias**... Pero no solo eso, ¡tiene dos islas llenas de caníbales! ¡Si caíamos ahí era nuestra perdición! Debíamos llegar a la orilla opuesta sin que cayéramos en algún obstáculo. No se quién tenía más miedo, si el Coronel o yo, pero no teníamos tiempo que perder, en 10 días salía mi vuelo hacia California y desde luego no podía perderlo, por mucho que me gustara la selva, no quería quedarme ni un minuto más del necesario en este sitio. Sin embargo, tal y como nos pasó en los dos desafíos anteriores, sabíamos algunos detalles acerca de este río. Las dos islas no podían estar en el mismo sitio que la orilla opuesta ni juntas en la misma casilla ni en la última ni primera fila. Además, gracias al tercer libro que me dejó Keeley, sabía que este río se podía modelar como un Proceso de Decisión de Márkov, así que apliqué lo que había aprendido para sobrevivir a estos peligros. El estado inicial era (0,0) y el estado objetivo era la orilla opuesta, situada, como las islas, en zonas del río desconocidas, ya que todo estaba lleno de niebla y no podía ver nada...

Esta vez no os dejó con Keeley, si no con el Coronel que fue de gran ayuda para superar todos los retos y situaciones que nos acontecieron a la hora de cruzar el río Nung, así como para obtener una estrategia (política óptima) en cada zona del río teniendo en cuenta corrientes, islas y demás obstáculos.

¡Hasta la próxima!

PROCESO DE DECISIÓN DE MARKOV

Para superar este reto, empezamos definiendo el entorno que nos rodeaba. La función `creación_entorno` que recibía como parámetros las filas, columnas y la semilla (nos indicará el mapa que tenemos que hacer). Esta función establece las islas de manera aleatoria asegurándose de que las filas no se encuentran ni en la primera ni en la última columna, que no están en la misma posición y que la posición no coincide con la posición de destino (que también es aleatoria). Devolvemos un diccionario con los datos necesario: filas, columnas, posición de origen, de destino y posición de las islas.

La función de visualizar el río recibe como argumentos el diccionario de entorno y la posición del agente. El Capitán no conoce la distribución de elementos, se muestra para facilitar la visualización del escenario al usuario, en cada una de las decisiones del Proceso de Decisión de Márkov.

La función `probabilidad_sig_estado` calcula la probabilidad de llegar a una celda objetivo ejecutando una acción desde una celda de origen. Recibe como argumentos la celda de origen, la de destino, la acción que queremos tomar, la lista de fuerzas del río y el diccionario de entorno. Teniendo en cuenta lo que nos dice el enunciado, la función de probabilidad viene definida por:

$$P(s' | s, a) = \begin{cases} p_{\text{dir}} = 1 - \text{river_strength}(j), & \text{si } s' \text{ es la casilla en la dirección de } a, a \neq \text{down} \\ p_{\text{dir}} = 1, & \text{si } a = \text{down y } s' \text{ es la casilla inferior} \\ p_{\text{down}} = \text{river_strength}(j), & \text{si } s' \text{ es la casilla inferior y } a \neq \text{down} \\ p_{\text{stay}} = 1, & \text{si } s' = s \text{ por un borde o una isla} \\ 0, & \text{en cualquier otro caso.} \end{cases}$$

Figura 1: Función de probabilidad siguiente estado

Por lo que para calcular dicha probabilidad analizaremos las fuerzas del río y la acción pedida y el destino al que llegaríamos ejecutando dicha acción. Si la celda de destino está fuera de los límites del río o coincide con una isla, la celda objetivo pasa a ser la celda actual. Calculamos las probabilidades teniendo en cuenta los arrastre hacia una casilla inferior (fila +1) y teniendo en cuenta si la acción es “D” (down) o si no lo es.

La función `fuerza_río` calcula la fuerza de arrastre en cada columna, recibiendo así los inputs columnas (nº de columnas) y semilla para identificar el mapa del río. La fuerza para las columnas de los bordes (primera y última) es 0, mientras que para las interiores es un valor aleatorio siguiendo una distribución uniforme entre 0.06 y 0.94.

La función `recompensas` simplemente recibe la celda actual y el diccionario de entorno y devuelve la recompensa correspondiente a la celda siento 100 si llegamos a la otra orilla, -100 si hay una isla y -1 para cualquier otro caso.

La función `informacion_actual` recibe como argumentos la celda de la que se quiere obtener la información, la lista de acciones disponibles, la lista de fuerzas del río y el diccionario de entorno. Imprime por pantalla la posición de la celda, las acciones disponibles en ese estado, las probabilidades de transición teniendo en cuenta bordes e islas (siguiendo lógica de la función anterior), el destino al que llega tomando una determinada acción (se evalúan todas las acciones posibles) y por último la recompensa obtenida en ese estado.

La función `mostrar_mapas` muestra, una vez ha recibido el diccionario de coordenadas el mapa correspondiente por pantalla.

La función `value_iteration` es la que lleva a cabo el algoritmo principal de este proceso. Recibe como argumentos el diccionario de entorno, la lista de fuerzas, una gamma dada que equivale al factor de descuento, una ϵ dada que será la condición de parada (verificará la perfección de la política). Comenzamos definiendo los mapas de utilidad y política óptima inicializados a 0 y "S" (por la acción S) respectivamente. Usamos estos valores porque son neutrales. Para trabajar con los mapas haremos una copia para evitar errores e iremos evaluando las posiciones del mapa de utilidad y viendo todas las opciones posibles de destino desde esa celda. Evaluaremos si la posición actual no es una isla ni es el destino, ya que, si no, no nos vale para la evaluación y pasaremos a evaluar cada acción posible en cada una de las opciones de destino desde la posición actual. Calcularemos la probabilidad de llegar al siguiente estado con la función `probabilidad_sig_estado` y si esa probabilidad es mayor que 0 estableceremos como estado a analizar la propia opción si es válida o la posición actual si está fuera de los límites del río. Calculamos la recompensa en dicho estado y aplicamos la **ecuación de Bellman**, buscando maximizar el valor de las acciones. Una vez obtenido ese máximo tomamos el valor anterior que tenía la posición en la copia del mapa de utilidad y calculamos la diferencia, ya que, tal y como hemos estudiado en clase, la condición de parada para un proceso de Markov es la homogeneidad de los valores, que o bien sean iguales, o prácticamente iguales, de ahí la utilidad de la ϵ . Una vez hemos actualizado las copias y los mapas reales comprobaremos si la diferencia del máximo actual y el valor anterior de dicha posición en el mapa de utilidad es menor que ϵ , en cuyo caso hemos llegado a lo que buscábamos y la condición de parada pasa a ser True, saliendo así del bucle y devolviendo los mapas de utilidad y política óptima.

Finalmente, podemos considerar como principales las funciones `proceso_completo` y `simulación`. La primera se encarga de encajar todas las piezas del puzzle, recibiendo como argumentos el nº de filas y columnas, la semilla, el factor de descuento gamma y la condición de parada ϵ . Comienza creando el entorno y obteniendo las fuerzas del río. Con el algoritmo Value Iteration obtiene el mapa de utilidad y de política óptima y usando la función `mostrar_mapas` los imprime por pantalla. Establecemos como estado actual el origen y establecemos un máximo de pasos que será útil para algunos pasos de la simulación (se explicará con detenimiento más adelante). Imprimimos también la visualización inicial del río. Comienza el bucle de búsqueda y ejecución. Las condiciones de este bucle son que la misión

no se cumpla y que el número de pasos sea menor al máximo. Obtenemos la acción a tomar en dicho estado y la fuerza correspondiente con la columna de la posición estado_actual. Imprimimos por pantalla la información de dicho estado usando información_actual. Para hacerlo más realista hemos establecido una condición aleatoria que establecerá corrientes si el valor aleatorio es inferior a la fuerza en dicha posición y la acción es distinta de “D” (down), en cuyo caso, Willard es arrastrado por la corriente y la acción final pasa a ser D, en caso contrario, consigue mantener el rumbo y la acción es la obtenida por el mapa de utilidad. Actualizamos el destino con los valores correspondientes a la acción dada y verificamos si esa posición entra dentro de los límites del río y no es una isla en cuyo caso estado_actual pasa a ser la posición de destino. En caso de choque, estado_actual no se modifica. Imprimimos el mapa del río con las actualizaciones pertinentes y evaluamos la recompensa acumulada hasta ese estado. Comprobamos si hemos llegado al destino, en cuyo caso devolvemos los pasos que han sido necesarios para completar la misión junto con su recompensa asociada y ponemos misión_en_curso a False.

Por otra parte, la función simulación contiene, como su propio nombre indica, distintas simulaciones variando datos para ver cómo reaccionaría el agente. Hemos podido deducir que a menor ϵ mayor perfección de la política, a menor γ , se produce un bucle infinito, no para (de ahí la utilidad del número máximo de pasos). Sin embargo, con un γ alto aseguramos la máxima puntuación. Por otro lado, cambiar la semilla cambia la posición de los elementos, es decir es un mapa distinto y cambiar filas o columnas, no afecta al desarrollo de las políticas, será necesario realizar más pasos, pero no afecta a lo que es el resultado. Podemos concluir:

GAMMA:

- Alto: asegura mayor puntuación al ser más prudente e ir evaluando con cuidado
- Bajo: bucle infinito, no se preocupa por la puntuación, cualquier celda le vale

ÉPSILON:

- Alto: el agente es más “vago” no asegura una política óptima, pero es más rápido
- Bajo: asegura una política óptima perfecta, aunque tarda más

SEMILLA:

- Cambiarla implica cambio de plano

FILAS Y COLUMNAS:

- Aumenta el número de pasos hasta el destino, pero no afecta al resultado ni a la obtención de políticas.

Visualizaciones

Antes de concluir, quiero mostrarte algunos de los mapas que hice de cada desafío de nuestra aventura.

PARTE 1 – GUIADA

```
Has escogido la opción guiada...¡Buena suerte! 🍀
¡Hola! Soy el Capitán Willard. Acompáñame en esta aventura para encontrar al Coronel Kurtz evitando los
obstáculos letales.
Partimos de la celda (0,0)...

['CW', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']

Tenemos el siguiente conocimiento acerca de la celda (0, 0)
Brisa: False
Ronquido: False
Resplandor: False
Pared arriba: True
Pared abajo: False
Pared izq: True
Pared der: False

¡Estamos en una celda segura, ni soldados ni precipicios!
```

Figura 2. Desarrollo de la primera parte guiada, perceptos y tablero visible

```

Celdas visitadas
- (0, 0)
- (0, 1)
- (0, 2)
- (0, 3)
- (1, 1)

Celdas seguras (5):
✓ (0, 0)
✓ (0, 1)
✓ (0, 2)
✓ (0, 3)
✓ (1, 1)

Información sobre la salida:
- Posibles ubicaciones de la salida:
  - (1, 0)
  - (0, 1)
  - (2, 1)
  - (1, 2)

Información sobre el soldado:
- No tenemos información sobre el soldado

Información sobre precipicios:
- No tenemos información sobre precipicios

```

Figura 3. Desarrollo de la primera parte guiada, celdas visitadas, seguras e información sobre estímulos salida, soldado y precipicios

PARTE 1 – AGENTE BFS

```

Moviendo de (4, 0) a (5, 0)
['X', '*', '*', '*', '*', '*']
['X', '*', '*', '*', '*', '*']
['X', '*', '*', '*', '*', '*']
['X', '*', '*', '*', '*', '*']
['X', '*', '*', '*', '*', '*']
['CW', '*', '*', '*', '*', '*']
Moviendo de (5, 0) a (5, 1)
['X', '*', '*', '*', '*', '*']
['X', '*', '*', '*', '*', '*']
['X', '*', '*', '*', '*', '*']
['X', '*', '*', '*', '*', '*']
['X', '*', '*', '*', '*', '*']
['X', 'CW', '*', '*', '*', '*']
Moviendo de (5, 1) a (4, 1)
['X', '*', '*', '*', '*', '*']
['X', '*', '*', '*', '*', '*']
['X', '*', '*', '*', '*', '*']
['X', '*', '*', '*', '*', '*']
['X', 'CW', '*', '*', '*', '*']
['X', 'X', '*', '*', '*', '*']
¡Hemos encontrado al Coronel Kurtz en (4, 1)!
-----
Ruta óptima para encontrar al Coronel desde la posición (0,0):
[(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1)]
-----
Ya tenemos al Coronel, ahora debemos buscar la salida...

```

Figura 4. Final de ejecución con ruta óptima para encontrar al Coronel desde el origen

```

Moviendo de (3, 2) a (4, 2)
['X', 'X', 'X', '*', '*', '*']
['X', 'X', 'X', '*', '*', '*']
['X', 'X', 'X', '*', '*', '*']
['X', 'X', 'X', '*', '*', '*']
['X', 'X', 'CW', '*', '*', '*']
['X', 'X', '*', '*', '*', '*']
Moviendo de (4, 2) a (5, 2)
¡Hemos encontrado la salida en (5, 2)!

¡VAMOS A LA SALIDA EN (5, 2)!
-----
Ruta óptima para encontrar la salida desde la posición (4, 1):
[(4, 1), (5, 1), (5, 2)]
-----
¡Lo lograste! Conseguimos escapar y convencer al Coronel Kurtz. ¡Muchas gracias por la ayuda!

```

Figura 5. Final de ejecución obtención de salida (ruta óptima) desde posición del Coronel

PARTE 2 – GUIADA

```

Recomendación: Celdas seguras [(0, 0), (2, 0), (1, 1)]

Movimientos posibles en la celda (1, 0): ['W', 'S', 'D']
Escoge un movimiento (W/A/S/D/G): D
¡Rumbo a la celda (1, 1)!
['X', '*', '*', '*', '*', '*']
['X', 'CW', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']

Recomendación: Celdas seguras [(0, 1), (1, 0)]

Movimientos posibles en la celda (1, 1): ['W', 'A', 'S', 'D']
Escoge un movimiento (W/A/S/D/G): 

```

Figura 6. Desarrollo de ejecución de parte guiada (similar a parte 1)

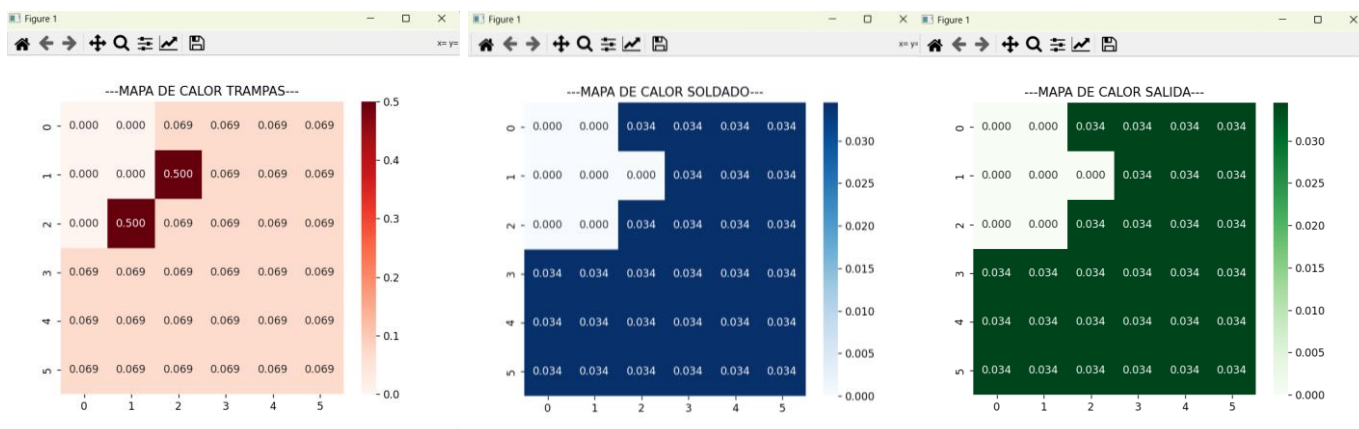


Figura 7. Mapas de calor de las trampas, soldado y salida durante un desarrollo de ejecución



Figura 8. Localización de elemento o probabilidades altas de dicho elemento

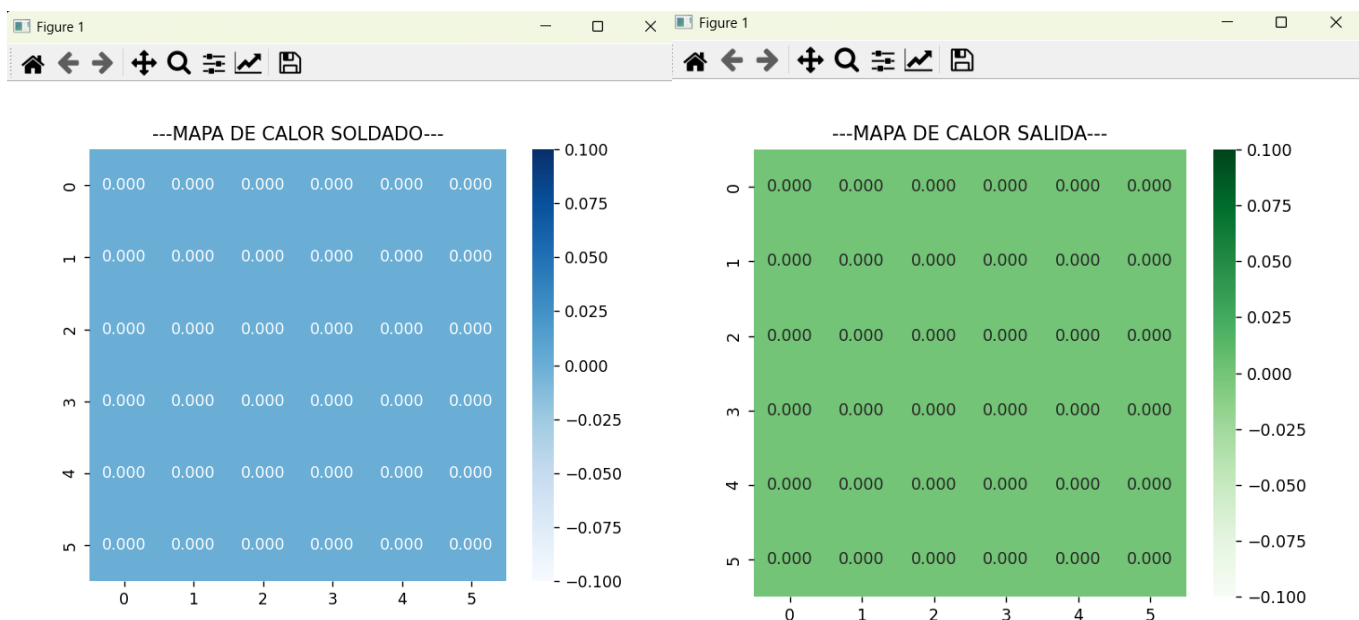


Figura 9. Soldado caído y salida encontrada

¡Rumbo a la celda (1, 1)!

```
['X', 'X', 'X', '*', '*', '*']
['X', 'CW', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']
```

¡Rumbo a la celda (1, 2)!

¡Hemos encontrado al Coronel Kurtz! Se hallaba escondido en la celda (1, 2)
Tenemos al coronel, pero debemos buscar la salida...

Figura 10. A* encuentra al Coronel y procede a buscar la salida

Vamos hacia (0, 4) -> Nos movemos a (1, 4)

```
['X', 'X', 'X', '*', '*', '*']
['X', 'X', 'X', 'X', 'CWCK', '*']
['*', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']
```

Vamos hacia (0, 4) -> Nos movemos a (0, 4)

¡Oh no! Te has encontrado con un obstáculo letal y has muerto
Muerte en la huida.Fallamos...

Figura 11. A* encuentra al Coronel, pero falla al buscar la salida

```

¡Hemos encontrado al Coronel Kurtz! Se hallaba escondido en la celda (3, 5)
Ya tenemos la salida: posición (2, 5). ¡Vamos hacia ahí! Misión cumplida
['X', 'X', 'X', 'X', 'X', 'X']
['X', 'X', 'X', 'X', 'X', '*']
['X', 'X', '*', 'X', 'X', 'X']
['X', 'X', 'X', 'X', 'X', 'CWCK']
['*', '*', '*', '*', '*', '*']
['*', '*', '*', '*', '*', '*']

Vamos hacia (2, 5) -> Nos movemos a (2, 5)

¡Encontramos la salida!
¡MISIÓN CUMPLIDA! Hemos conseguido escapar.

```

Figura 12. A* encuentra al Coronel, y obtiene la salida cumpliendo la misión

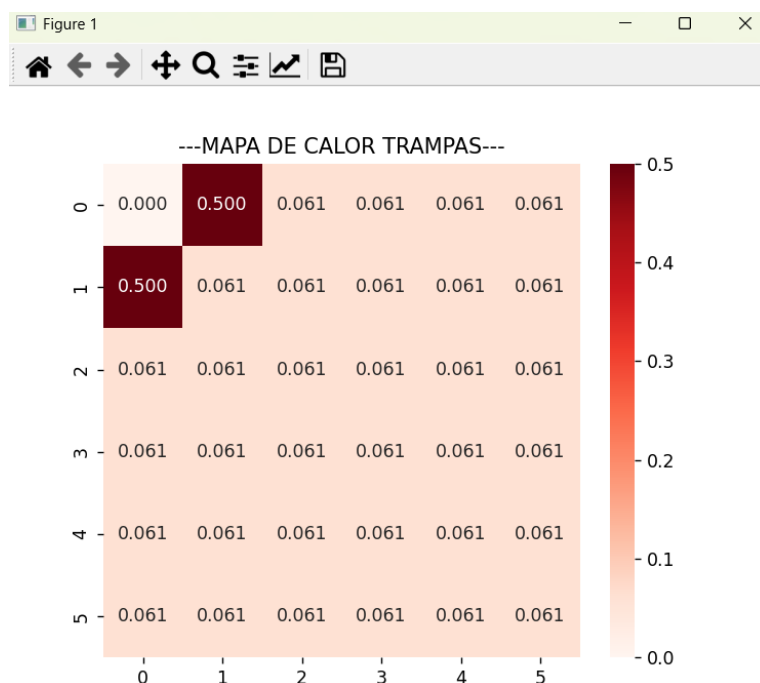


Figura 13. Situación insegura para el agente, se arriesgará sin certeza de éxito

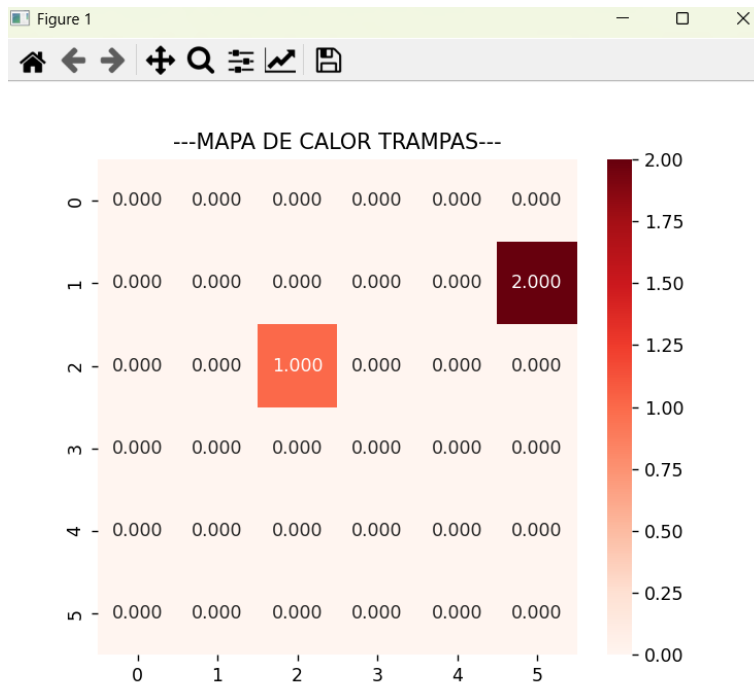


Figura 14. Dos trampas se encuentran en la misma celda (tablero de listas)

PARTE 3 – PROCESOS DE DECISIÓN DE MARKOV

```

Acciones disponibles en este estado: ['U', 'D', 'L', 'R', 'S']
Destino tomando la acción U: (0, 0) (Choque)
Probabilidades de transición: 1.0 de éxito, 0.0 de arrastre, 1.0 de choque
Destino tomando la acción D: (0, 0) (Choque)
Probabilidades de transición: 1.0 de éxito, 0.0 de arrastre, 1.0 de choque
Destino tomando la acción L: (0, 0) (Choque)
Probabilidades de transición: 1.0 de éxito, 0.0 de arrastre, 1.0 de choque
Destino tomando la acción R: (0, 1)
Probabilidades de transición: 1.0 de éxito, 0.0 de arrastre, 0.0 de choque
Destino tomando la acción S: (0, 0)
Probabilidades de transición: 1.0 de éxito, 0.0 de arrastre, 0.0 de choque
Recompensa obtenida en este estado: -1
Hemos conseguido mantener el rumbo!
CHOQUE! Willard rebota y permanece en (0, 0)
ACTUALIZACIÓN VISUALIZACIÓN
CWCK|| R || R || R ||
I || R || R || R ||
  || R || R || R || |
  || I || R || R ||
  || R || R || R ||
  || R || R || R || E |
  || R || R || R ||

-----INFORMACIÓN SOBRE (0, 0)-----
Posición actual: (0, 0)
Acciones disponibles en este estado: ['U', 'D', 'L', 'R', 'S']
Destino tomando la acción U: (0, 0) (Choque)
Probabilidades de transición: 1.0 de éxito, 0.0 de arrastre, 1.0 de choque
Destino tomando la acción D: (0, 0) (Choque)
Probabilidades de transición: 1.0 de éxito, 0.0 de arrastre, 1.0 de choque
Destino tomando la acción L: (0, 0) (Choque)
Probabilidades de transición: 1.0 de éxito, 0.0 de arrastre, 1.0 de choque

```

Figura 15. Escenario A, bucle infinito (hasta max_step) por Gamma bajo

```

-----INFORMACIÓN SOBRE (0, 4)-----
· Posición actual: (0, 4)
· Acciones disponibles en este estado: ['U', 'D', 'L', 'R', 'S']
· Destino tomando la acción U: (0, 4) (Choque)
· Probabilidades de transición: 0.7 de éxito, 0.3 de arrastre, 1.0 de choque
· Destino tomando la acción D: (1, 4)
· Probabilidades de transición: 1.0 de éxito, 0.0 de arrastre, 0.0 de choque
· Destino tomando la acción L: (0, 3)
· Probabilidades de transición: 0.7 de éxito, 0.3 de arrastre, 0.0 de choque
· Destino tomando la acción R: (0, 5)
· Probabilidades de transición: 0.7 de éxito, 0.3 de arrastre, 0.0 de choque
· Destino tomando la acción S: (0, 4)
· Probabilidades de transición: 0.7 de éxito, 0.3 de arrastre, 0.0 de choque
· Recompensa obtenida en este estado: -1
¡Hemos conseguido mantener el rumbo!
ACTUALIZACIÓN VISUALIZACIÓN
|  |  | R |  | R |  | R |  | R |  | CWCK |
|  |  | R |  | R |  | R |  | R |  | I  |
|  |  | I |  | R |  | R |  | R |  |  |
|  |  | R |  | R |  | R |  | R |  |  |
|  |  | R |  | R |  | R |  | R |  |  |
¡Misión cumplida! Pasos: 19
Recompensa total acumulada: 82

```

Figura 15. Escenario B, asegura máxima puntuación por Gamma alto

```

-----INFORMACIÓN SOBRE (1, 4)-----
· Posición actual: (1, 4)
· Acciones disponibles en este estado: ['U', 'D', 'L', 'R', 'S']
· Destino tomando la acción U: (0, 4)
· Probabilidades de transición: 1.0 de éxito, 0.0 de arrastre, 0.0 de choque
· Destino tomando la acción D: (2, 4)
· Probabilidades de transición: 1.0 de éxito, 0.0 de arrastre, 0.0 de choque
· Destino tomando la acción L: (1, 3)
· Probabilidades de transición: 1.0 de éxito, 0.0 de arrastre, 0.0 de choque
· Destino tomando la acción R: (1, 4) (Choque)
· Probabilidades de transición: 1.0 de éxito, 0.0 de arrastre, 1.0 de choque
· Destino tomando la acción S: (1, 4)
· Probabilidades de transición: 1.0 de éxito, 0.0 de arrastre, 0.0 de choque
· Recompensa obtenida en este estado: -1
¡Hemos conseguido mantener el rumbo!
ACTUALIZACIÓN VISUALIZACIÓN
|  |  | R |  | R |  | R |  | CWCK |
|  |  | R |  | R |  | R |  |  |
| I |  | R |  | I |  | R |  |  |
|  |  | R |  | R |  | R |  |  |
¡Misión cumplida! Pasos: 8
Recompensa total acumulada: 93

```

Figura 16. Escenario C, distinta semilla = distinto río

Final del viaje

Me hallo sentado en el avión de vuelta a California. A mi lado se encuentran el Coronel Kurtz y Keeley. Lo hemos conseguido. Nos espera un día de viaje así que decido dormir lo que en este mes no he dormido. Cierro los ojos y me imagino que estoy tranquilamente en mi hamaca de confianza en mi porche observando el mar y escuchando el sonido del silencio. Qué paz. Cuando abro los ojos veo que estoy en mi jardín y las olas rompen en el mar que tengo en frente. Menudo sueño más raro he tenido. Parecía tan real... Vuelvo a casa y enciendo la tele y de pronto veo cómo ha empezado la guerra contra Vietnam y cómo Estados Unidos juega con ventaja por el conocimiento de ciertas estrategias secretas que cierto Coronel dio a conocer al ejército. Así que no lo he soñado, es real, wow. Me pellizco porque sigo sin creérmelo del todo, pero es así, prueba de ello son las noticias que estoy viendo ahora mismo. Supongo que al final el futuro del país sí que dependía de mí...

1 mes después

Vuelta a la rutina, vuelta a las llamadas molestas de la oficina y yo a ignorarlas. Decido salir al porche a cultivar el huerto y a regar las plantas, cuando de pronto escucho algo raro viniendo del nogal. Esta vez no es un serrucho, es un sonido más bien metálico, como un tintíneo. Me acerco a ver qué es y como si no hubiera aprendido de la vez anterior otra nuez me vuelve a caer en la cabeza y con ella, otro papelito. Con sorna, lo abro y leo su contenido: "Hola Capitán. Espero que este mes le haya dado fuerzas para lo que se viene. Le espero en el parque del reloj a las 18:00. Keeley".

Me río, aunque ya sé lo que voy a hacer. Supongo que me toca hacer las maletas.

¡Hasta la próxima!